

ARRAY DATA STRUCTURE

Unit 3

Presented By: Saroj Bhandari

Syllabus

3.1 Introduction to Arrays - array representation

3.2 Advantages and Drawbacks of Arrays

3.2 sorting algorithms with efficiency: Bubble sort, Selection sort, Insertion sort, Merge sort, Quick Sort, Heap Sort, Radix sort, Bucket Sort, Concept of stable and unstable sorting

3.3 Searching Algorithms: Linear Search, Binary Search

Introduction to Sorting Algorithm

- A **Sorting Algorithm** is used to rearrange a given array or list of elements in an order. For example, a given array [10, 20, 5, 2] becomes [2, 5, 10, 20] after sorting in increasing order and becomes [20, 10, 5, 2] after sorting in decreasing order.
- **Sorting** refers to rearrangement of a given array or list of elements according to a comparison operator on the elements. The comparison operator is used to decide the new order of elements in the respective data structure.



Introduction to Array

- **Array** is a collection of items of the same variable type that are stored at contiguous memory locations. It is one of the most popular and simple data structures used in programming.

Basic terminologies of Array

- **Array Index:** In an array, elements are identified by their indexes. Array index starts from 0.
- **Array element:** Elements are items stored in an array and can be accessed by their index.
- **Array Length:** The length of an array is determined by the number of elements it can contain.

Memory representation of Array

- In an array, all the elements are stored in contiguous memory locations. So, if we initialize an array, the elements will be allocated sequentially in memory. This allows for efficient access and manipulation of elements.

int arr[] = {11, 9, 17, 89, 1, 90, 19, 5, 3, 23, 43, 99}

Address											
200	204	208	212	216	220	224	228	232	236	240	244
11	9	17	89	1	90	19	5	3	23	43	99
0	1	2	3	4	5	6	7	8	9	10	11
Index											

Declaration and Initialization of Array

```
// This array will store integer type element  
int arr[5];  
  
// This array will store char type element  
char arr[10];  
  
// This array will store float type element  
float arr[20];
```

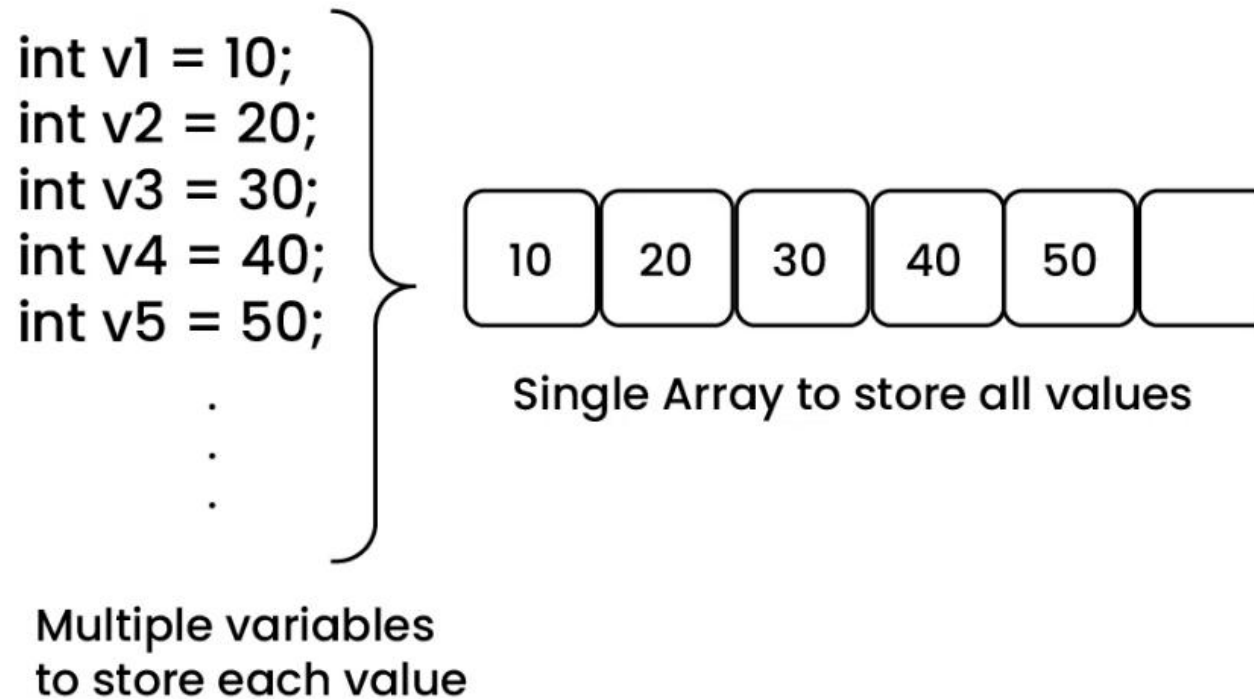
```
int arr[] = { 1, 2, 3, 4, 5 };  
char arr[5] = { 'a', 'b', 'c', 'd', 'e' };  
float arr[10] = { 1.4, 2.0, 24, 5.0, 0.0 };
```

Why do we **Need Arrays**?

- Assume there is a class of five students and if we have to keep records of their marks in examination then, we can do this by declaring five variables individual and keeping track of records but what if the number of students becomes very large, it would be challenging to manipulate and maintain the data.
- What it means is that, we can use normal variables (v1, v2, v3, ..) when we have a small number of objects. But if we want to store a large number of instances, it becomes difficult to manage them with normal variables.
- *The idea of an array is to represent many instances in one variable.*

Importance of Array

Importance of Array



Advantage of Array

- In an array, **accessing an element** is very easy by using the index number.
- The search process can be applied to an array easily.
- 2D Array is used to represent matrices.
- For any reason a user wishes to store multiple values of similar type then the Array can be used and utilized efficiently.
- Arrays have low overhead.
- C provides a set of built-in functions for manipulating arrays, such as sorting and searching.
- C supports arrays of multiple dimensions, which can be useful for representing complex data structures like matrices.
- Arrays can be easily converted to pointers, which allows for passing arrays to functions as arguments or returning arrays from functions.

Disadvantage of Array

- **Array size is fixed:** The array is static, which means its size is always fixed. The memory which is allocated to it cannot be increased or decreased.
- **Array is homogeneous:** The array is homogeneous, i.e., only one type of value can be store in the array. For example, if an array type "**int**", can only store integer elements and cannot allow the elements of other types such as double, float, char so on.
- **Array is Contiguous blocks of memory:** The array stores data in contiguous(one by one) memory location.
- **Insertion and deletion are not easy in Array:** The operation insertion and deletion over an array are problematic as to insert or delete anywhere in the array, it is necessary to traverse the array and then shift the remaining elements as per the operation.

Why Sorting Algorithms are Important?

- The **sorting algorithm** is important in Computer Science because it reduces the complexity of a problem.
- There is a wide range of applications for these algorithms, including searching algorithms, database algorithms, divide and conquer methods, and data structure algorithms.
- Some **important scientific applications** where sorting algorithms are used :
 - When you have **hundreds of datasets** you want to print, you might want to arrange them in some way.
 - Once we get the data sorted, we can get the k-th smallest and k-th largest item in **$O(1)$** time.
 - Searching any element in a **huge data set becomes easy**. We can use Binary search method for search if we have sorted data. So, Sorting become important here.
 - They can be used in software and in **conceptual problems** to solve more advanced problems.

Types Of Sorting

1. Internal Sorting :

- When all data is placed in the **main memory** or **internal memory** then sorting is called internal sorting.
- In internal sorting, the problem cannot take input beyond its size.
- Example: **heap sort, bubble sort, selection sort, quick sort, shell sort, insertion sort.**

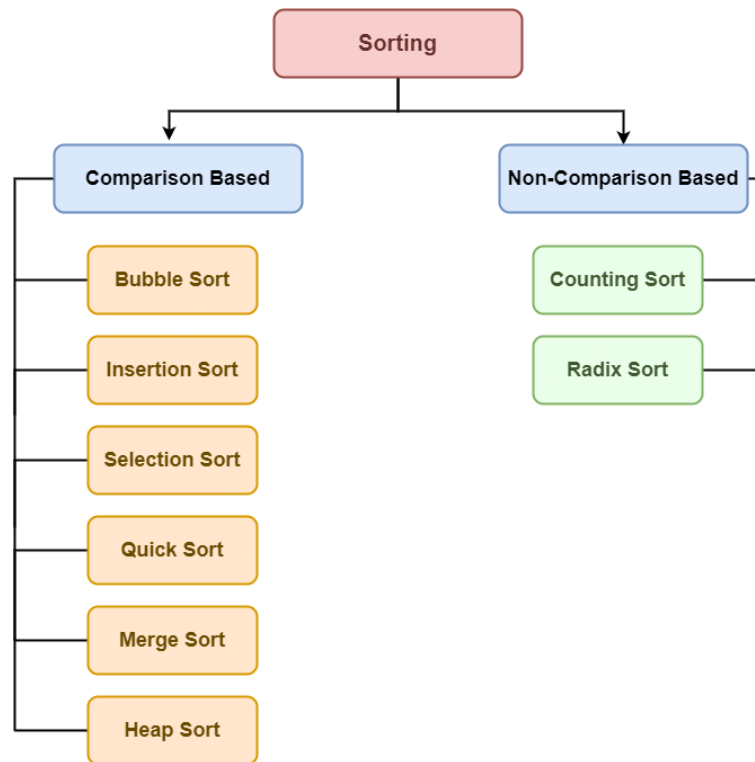
2. External Sorting :

- When all data that needs to be sorted cannot be placed in memory at a time, the sorting is called external sorting.
- External Sorting is used for the massive amount of data.
- Merge Sort and its variations are typically used for external sorting.
- Some external storage like hard disks and CDs are used for external sorting.
- Example: Merge sort, Tag sort, Polyphase sort, Four tape sort, External radix sort, Internal merge sort, etc.

Types of Sorting Techniques

- There are various sorting algorithms are used in data structures.
- The following two types of sorting algorithms can be broadly classified:
 - **Comparison-based:** We compare the elements in a comparison-based sorting algorithm)
 - **Non-comparison-based:** We do not compare the elements in a non-comparison-based sorting algorithm)

Types of Sorting Techniques



Bubble Sort Algorithm

- **Bubble Sort** is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order.
- This algorithm is not suitable for large data sets as its average and worst-case time complexity are quite high.

Bubble Sort

- https://www.youtube.com/watch?v=Twpo1-8--6M&ab_channel=VisualHow

How does Bubble Sort Work?

03
Step

Placing 3rd largest element at its correct position



Bubble sort



3 / 3

Code

```
1 // Optimized implementation of Bubble sort
2 #include <stdbool.h>
3 #include <stdio.h>
4
5 void swap(int* xp, int* yp){
6     int temp = *xp;
7     *xp = *yp;
8     *yp = temp;
9 }
10
11 // An optimized version of Bubble Sort
12 void bubbleSort(int arr[], int n){
13     int i, j;
14     bool swapped;
15     for (i = 0; i < n - 1; i++) {
16         swapped = false;
17         for (j = 0; j < n - i - 1; j++) {
18             if (arr[j] > arr[j + 1]) {
19                 swap(&arr[j], &arr[j + 1]);
20                 swapped = true;
21             }
22         }
23     }
```

```
24         // If no two elements were swapped by inner loop,
25         // then break
26         if (swapped == false)
27             break;
28     }
29 }
30
31 // Function to print an array
32 void printArray(int arr[], int size){
33     int i;
34     for (i = 0; i < size; i++)
35         printf("%d ", arr[i]);
36 }
37
38 int main(){
39     int arr[] = { 64, 34, 25, 12, 22, 11, 90 };
40     int n = sizeof(arr) / sizeof(arr[0]);
41     bubbleSort(arr, n);
42     printf("Sorted array: \n");
43     printArray(arr, n);
44     return 0;
45 }
```

Complexity Analysis of Bubble Sort:

- **Time Complexity:** $O(n^2)$
- **Auxiliary Space:** $O(1)$
- **Advantages of Bubble Sort:**
 - Bubble sort is easy to understand and implement.
 - It does not require any additional memory space.
 - It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.
- **Disadvantages of Bubble Sort:**
 - Bubble sort has a time complexity of $O(n^2)$ which makes it very slow for large data sets.
 - Bubble sort has almost no or limited real world applications. It is mostly used in academics to teach different ways of sorting.

Selection Sort

- **Selection Sort** is a comparison-based sorting algorithm.
- It sorts an array by repeatedly selecting the **smallest (or largest)** element from the unsorted portion and swapping it with the first unsorted element.
- This process continues until the entire array is sorted.
 - First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.
 - Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.
 - We keep doing this until we get all elements moved to correct position.

Example

06
Step

We get the sorted array at the end.

arr[] =

11	12	22	25	64
----	----	----	----	----

Sorted array

Selection Sort Algorithm



6 / 6

Example Code

```
1 // C program for implementation of selection sort
2 #include <stdio.h>
3
4 void selectionSort(int arr[], int n) {
5     for (int i = 0; i < n - 1; i++) {
6
7         // Assume the current position holds
8         // the minimum element
9         int min_idx = i;
10
11        // Iterate through the unsorted portion
12        // to find the actual minimum
13        for (int j = i + 1; j < n; j++) {
14            if (arr[j] < arr[min_idx]) {
15
16                // Update min_idx if a smaller element is found
17                min_idx = j;
18            }
19        }
20
21        // Move minimum element to its
22        // correct position
23        int temp = arr[i];
24        arr[i] = arr[min_idx];
25        arr[min_idx] = temp;
26    }
27 }
28
```

```
29 void printArray(int arr[], int n) {
30     for (int i = 0; i < n; i++) {
31         printf("%d ", arr[i]);
32     }
33     printf("\n");
34 }
35
36 int main() {
37     int arr[] = {64, 25, 12, 22, 11};
38     int n = sizeof(arr) / sizeof(arr[0]);
39
40     printf("Original array: ");
41     printArray(arr, n);
42
43     selectionSort(arr, n);
44
45     printf("Sorted array: ");
46     printArray(arr, n);
47
48     return 0;
49 }
```

Complexity Analysis of Selection Sort

Time Complexity: $O(n^2)$,as there are two nested loops:

- One loop to select an element of Array one by one = $O(n)$
- Another loop to compare that element with every other Array element = $O(n)$
- Therefore overall complexity = $O(n) * O(n) = O(n*n) = O(n^2)$

Auxiliary Space: $O(1)$ as the only extra memory used is for temporary variables.

Advantages of Selection Sort

- Easy to understand and implement, making it ideal for teaching basic sorting concepts.
- Requires only a constant $O(1)$ extra memory space.
- It requires less number of swaps (or memory writes) compared to many other standard algorithms. Only cycle sort beats it in terms of memory writes. Therefore it can be simple algorithm choice when memory writes are costly.

Disadvantages of the Selection Sort

- Selection sort has a time complexity of $O(n^2)$ makes it slower compared to algorithms like Quick Sort or Merge Sort.
- Does not maintain the relative order of equal elements which means it is not stable.

Applications of Selection Sort

- Perfect for teaching fundamental sorting mechanisms and algorithm design.
- Suitable for small lists where the overhead of more complex algorithms isn't justified and memory writing is costly as it requires less memory writes compared to other standard sorting algorithms.
- Heap Sort algorithm is based on Selection Sort.

Insertion Sort Algorithm

- **Insertion sort** is a simple sorting algorithm that works **by iteratively inserting each element** of an unsorted list into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands. You split the cards into two groups: the sorted cards and the unsorted cards. Then, you pick a card from the unsorted group and put it in the right place in the sorted group.
- We start with the second element of the array as the **first element is assumed to be sorted**.
- **Compare the second element with the first element** if the second element is smaller then swap them.
- **Move to the third element**, compare it with the first two elements, and put it in its correct position
- **Repeat until the entire array is sorted.**

Example

Sorted Array

The sorted part contains the whole array. Means that the whole array is sorted.

arr[] =

5	6	11	12	13
---	---	----	----	----

Sorted

Insertion Sort



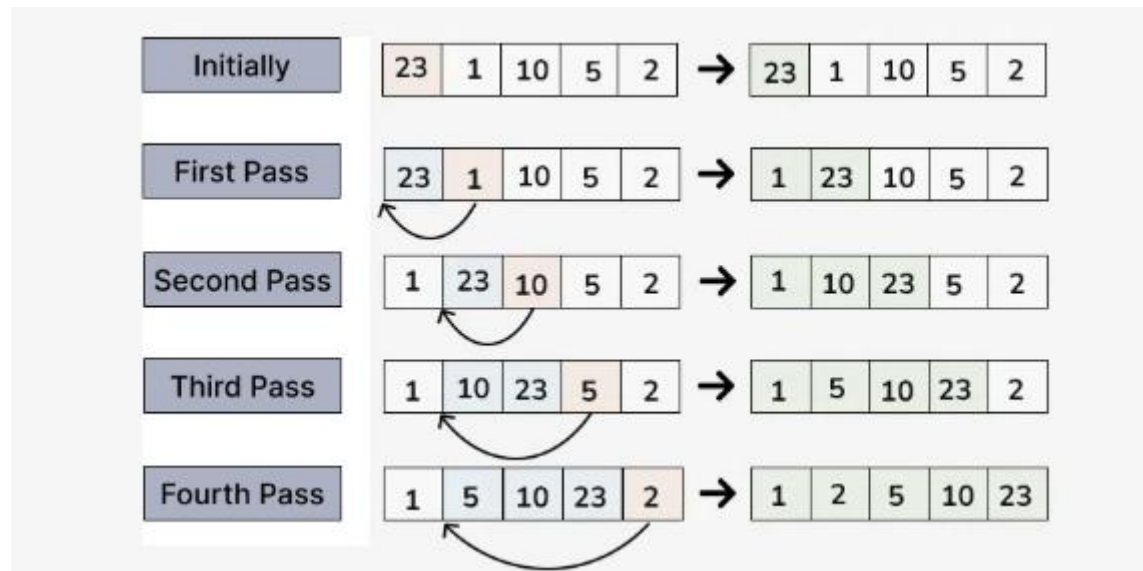
5 / 5

Code

```
1 // C program for implementation of Insertion Sort
2 #include <stdio.h>
3
4 /* Function to sort array using insertion sort */
5 void insertionSort(int arr[], int n)
6 {
7     for (int i = 1; i < n; ++i) {
8         int key = arr[i];
9         int j = i - 1;
10
11         /* Move elements of arr[0..i-1], that are
12            greater than key, to one position ahead
13            of their current position */
14         while (j >= 0 && arr[j] > key) {
15             arr[j + 1] = arr[j];
16             j = j - 1;
17         }
18         arr[j + 1] = key;
19     }
20 }
21
22 /* A utility function to print array of size n */
23 void printArray(int arr[], int n)
24 {
25     for (int i = 0; i < n; ++i)
26         printf("%d ", arr[i]);
27     printf("\n");
28 }
```

```
29
30 // Driver method
31 int main()
32 {
33     int arr[] = { 12, 11, 13, 5, 6 };
34     int n = sizeof(arr) / sizeof(arr[0]);
35
36     insertionSort(arr, n);
37     printArray(arr, n);
38
39     return 0;
40 }
41
```

Illustration



Output 5 6 11 12 13

Complexity Analysis of Insertion Sort

- **Time Complexity**
 - **Best case:** $O(n)$, If the list is already sorted, where n is the number of elements in the list.
 - **Average case:** $O(n^2)$, If the list is randomly ordered
 - **Worst case:** $O(n^2)$, If the list is in reverse order
- **Space Complexity**
 - **Auxiliary Space:** $O(1)$, Insertion sort requires **$O(1)$** additional space, making it a space-efficient sorting algorithm.

Advantages and Disadvantages of Insertion Sort

- **Advantages**

- Simple and easy to implement.
- **Stable** sorting algorithm.
- Efficient for small lists and nearly sorted lists.
- Space-efficient as it is an in-place algorithm.
- Adoptive. the number of inversions is directly proportional to number of swaps. For example, no swapping happens for a sorted array and it takes $O(n)$ time only.

- **Disadvantages**

- Inefficient for large lists.
- Not as efficient as other sorting algorithms (e.g., merge sort, quick sort) for most cases.

Applications of Insertion Sort

- Insertion sort is commonly used in situations where:
- The list is small or nearly sorted.
- Simplicity and stability are important.
- Used as a subroutine in Bucket Sort
- Can be useful when array is already almost sorted (very few inversions)
- Since Insertion sort is suitable for small sized arrays, it is used in Hybrid Sorting algorithms along with other efficient algorithms like Quick Sort and Merge Sort. When the subarray size becomes small, we switch to insertion sort in these recursive algorithms. For example IntroSort and TimSort use insertions sort.

Introduction to **Exchange Sort** Algorithm

- *Exchange sort is an algorithm used to sort in **ascending** as well as **descending** order. It compares the **first** element with every element if any element seems out of order it swaps.*
- **Example:**
- ***Input:** $arr[] = \{5, 1, 4, 2, 8\}$
Output: $\{1, 2, 4, 5, 8\}$
Explanation: Working of exchange sort:*

Example

- **1st Pass:**
Exchange sort starts with the very first elements, comparing with other elements to check which one is greater.
(5 1 4 2 8) $\rightarrow$ (1 5 4 2 8).
Here, the algorithm compares the first two elements and swaps since $5 > 1$.
No swap since none of the elements is smaller than 1 so after 1st iteration (1 5 4 2 8)
- **2nd Pass:**
(1 5 4 2 8) $\rightarrow$ (1 4 5 2 8), since $4 < 5$
(1 4 5 2 8) $\rightarrow$ (1 2 5 4 8), since $2 < 4$
(1 2 5 4 8) No change since in this there is no other element smaller than 2
- **3rd Pass:**
(1 2 5 4 8) $\rightarrow$ (1 2 4 5 8), since $4 < 5$
after completion of the iteration, we found array is sorted
- *After completing the iteration it will come out of the loop, Therefore array is sorted.*

To sort in **Ascending** order:

```
procedure ExchangeSort(num: list of sortable items)
  n = length(A)

  // outer loop
  for i = 1 to n - 2 do

    // inner loop

    for j = i + 1 to n-1 do

      if num[i] > num[j] do

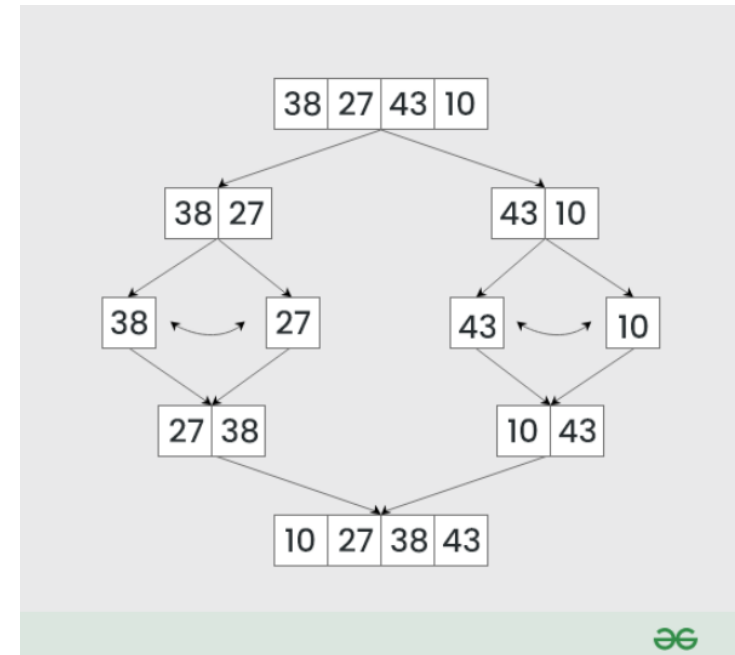
        swap(num[i], num[j])
      end if
    end for
  end for
end procedure
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
Auxiliary Space : $O(1)$
- **Advantages** of using **Exchange sort** over other sorting methods:
 - There are some situations where exchange sort may be preferable over other algorithms. For example, exchange sort may be useful when sorting very small arrays or when sorting data that is already mostly sorted. In these cases, the overhead of implementing a more complex algorithm may not be worth the potential performance gains.
 - Another advantage of the exchange sort is that it is stable, meaning that it preserves the relative order of equal elements. This can be important in some applications, such as when sorting records that contain multiple fields.

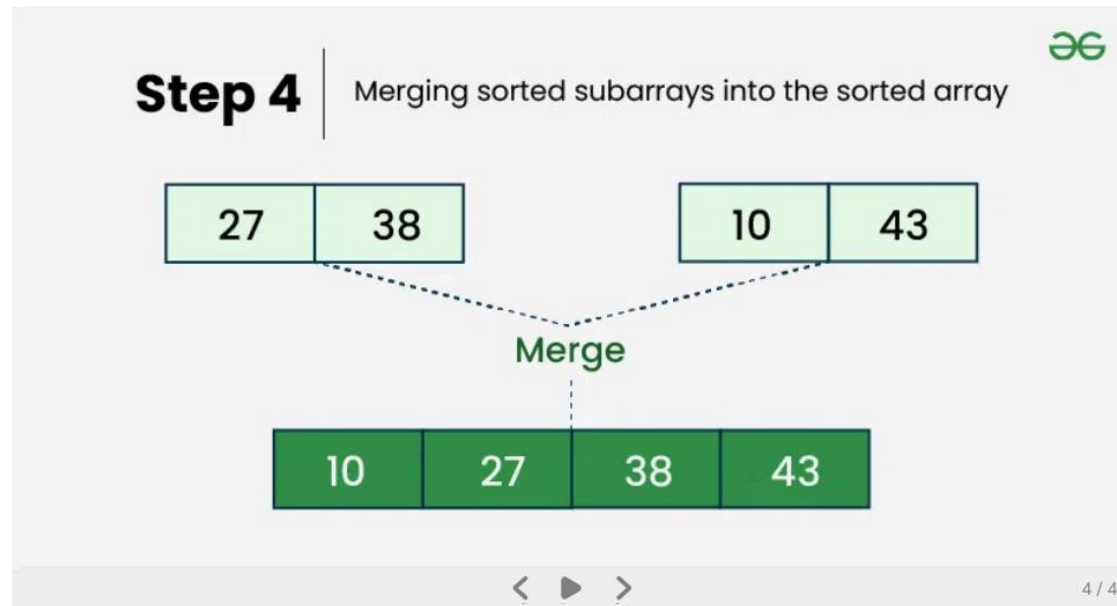
Merge Sort

- **Merge sort** is a popular sorting algorithm known for its efficiency and stability. It follows the divide-and-conquer approach.
- It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.



How does Merge Sort work?

- Illustration of Merge Sort:
- Let's sort the array or list [38, 27, 43, 10] using Merge Sort



Example Explanation

- *Let's look at the working of above example:*
- **Divide:**
 - *[38, 27, 43, 10] is divided into [38, 27] and [43, 10].*
 - *[38, 27] is divided into [38] and [27].*
 - *[43, 10] is divided into [43] and [10].*
- **Conquer:**
 - *[38] is already sorted.*
 - *[27] is already sorted.*
 - *[43] is already sorted.*
 - *[10] is already sorted.*
- **Merge:**
 - *Merge [38] and [27] to get [27, 38].*
 - *Merge [43] and [10] to get [10, 43].*
 - *Merge [27, 38] and [10, 43] to get the final sorted list [10, 27, 38, 43]*
- *Therefore, the sorted list is [10, 27, 38, 43].*

Code Implementation

```
1 // C program for Merge Sort
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 // Merges two subarrays of arr[].
6 // First subarray is arr[l..m]
7 // Second subarray is arr[m+1..r]
8 void merge(int arr[], int l, int m, int r)
9 {
10     int i, j, k;
11     int n1 = m - l + 1;
12     int n2 = r - m;
13
14     // Create temp arrays
15     int L[n1], R[n2];
16
17     // Copy data to temp arrays L[] and R[]
18     for (i = 0; i < n1; i++)
19         L[i] = arr[l + i];
20     for (j = 0; j < n2; j++)
21         R[j] = arr[m + 1 + j];
22
23     // Merge the temp arrays back into arr[l..r]
24     i = 0;
25     j = 0;
26     k = l;
27     while (i < n1 && j < n2) {
28         if (L[i] <= R[j]) {
29             arr[k] = L[i];
30             i++;
31         }
```

```
32     else {
33         arr[k] = R[j];
34         j++;
35     }
36     k++;
37 }
38
39 // Copy the remaining elements of L[],
40 // if there are any
41 while (i < n1) {
42     arr[k] = L[i];
43     i++;
44     k++;
45 }
46
47 // Copy the remaining elements of R[],
48 // if there are any
49 while (j < n2) {
50     arr[k] = R[j];
51     j++;
52     k++;
53 }
54 }
```

```
55
56 // l is for left index and r is right index of the
57 // sub-array of arr to be sorted
58 void mergeSort(int arr[], int l, int r)
59 {
60     if (l < r) {
61         int m = l + (r - l) / 2;
62
63         // Sort first and second halves
64         mergeSort(arr, l, m);
65         mergeSort(arr, m + 1, r);
66
67         merge(arr, l, m, r);
68     }
69 }
70
71 // Function to print an array
72 void printArray(int A[], int size)
73 {
74     int i;
75     for (i = 0; i < size; i++)
76         printf("%d ", A[i]);
77     printf("\n");
78 }
79 }
```

Code Implementation

```
70 -
71 // Function to print an array
72 void printArray(int A[], int size)
73 {
74     int i;
75     for (i = 0; i < size; i++)
76         printf("%d ", A[i]);
77     printf("\n");
78 }
79
80 // Driver code
81 int main()
82 {
83     int arr[] = { 12, 11, 13, 5, 6, 7 };
84     int arr_size = sizeof(arr) / sizeof(arr[0]);
85
86     printf("Given array is \n");
87     printArray(arr, arr_size);
88
89     mergeSort(arr, 0, arr_size - 1);
90
91     printf("\nSorted array is \n");
92     printArray(arr, arr_size);
93     return 0;
94 }
```

Complexity Analysis of Merge Sort

- **Time Complexity:**
 - **Best Case:** $O(n \log n)$, When the array is already sorted or nearly sorted.
 - **Average Case:** $O(n \log n)$, When the array is randomly ordered.
 - **Worst Case:** $O(n \log n)$, When the array is sorted in reverse order.
- **Auxiliary Space:** $O(n)$, Additional space is required for the temporary array used during merging.

Applications of Merge Sort:

- ❑ Sorting large datasets
- ❑ External sorting (when the dataset is too large to fit in memory)
- ❑ Inversion counting
- ❑ Merge Sort and its variations are used in library methods of programming languages.
 - ❑ Its variation TimSort is used in Python, Java Android and Swift. The main reason why it is preferred to sort non-primitive types is stability which is not there in QuickSort.
 - ❑ Arrays.sort in Java uses QuickSort while Collections.sort uses MergeSort.
- ❑ It is a preferred algorithm for sorting Linked lists.
- ❑ It can be easily parallelized as we can independently sort subarrays and then merge.
- ❑ The merge function of merge sort to efficiently solve the problems like union and intersection of two sorted arrays.

Advantages and Disadvantages of Merge Sort

- **Advantages**

- **Stability** : Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
- **Guaranteed worst-case performance**: Merge sort has a worst-case time complexity of $O(N \log N)$, which means it performs well even on large datasets.
- **Simple to implement**: The divide-and-conquer approach is straightforward.
- **Naturally Parallel** : We independently merge subarrays that makes it suitable for parallel processing.

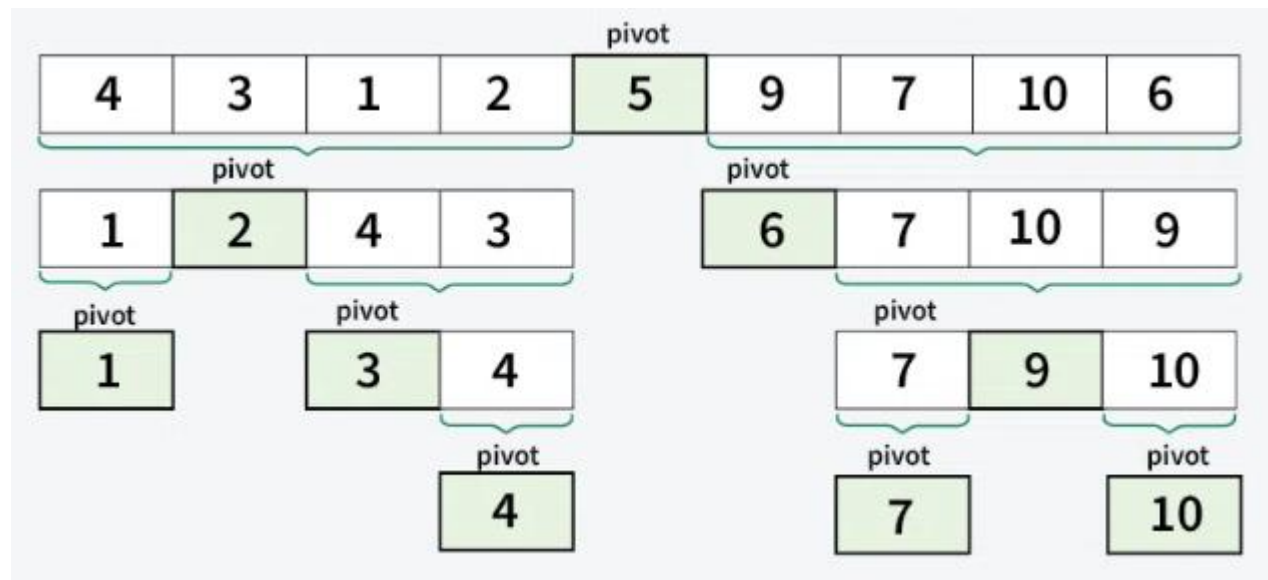
- **Disadvantages**

- **Space complexity**: Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
- **Not in-place**: Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- Merge Sort is **Slower than QuickSort in general** as QuickSort is more cache friendly because it works in-place.

Quick Sort

- **QuickSort** is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.
- It works on the principle of **divide and conquer**, breaking down the problem into smaller sub-problems.
- There are mainly three steps in the algorithm:
 - **Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
 - **Partition the Array:** Rearrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right. The pivot is then in its correct position, and we obtain the index of the pivot.
 - **Recursively Call:** Recursively apply the same process to the two partitioned sub-arrays (left and right of the pivot).
 - **Base Case:** The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

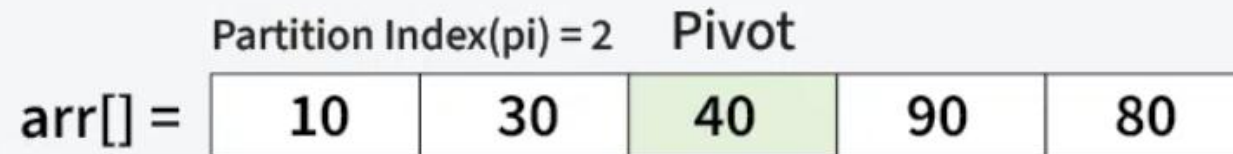
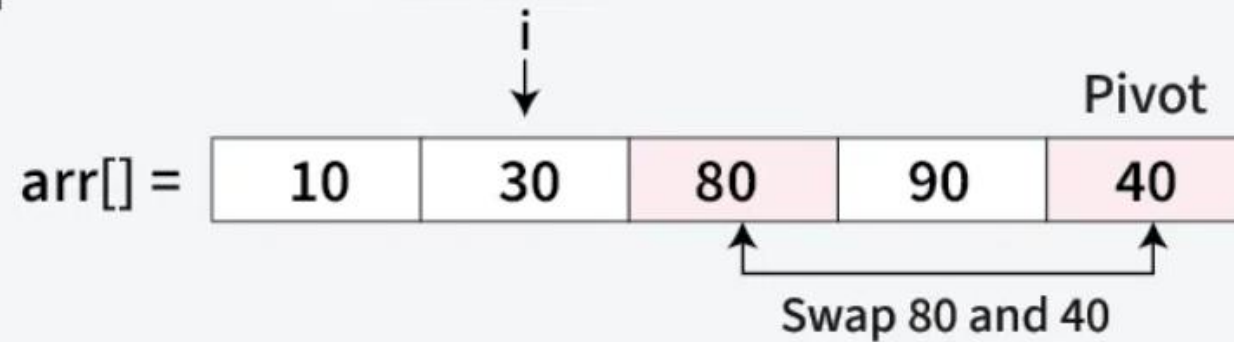
Basic overview of how the **QuickSort** algorithm works.



Illustration

06
Step

Since traversal of j has ended. Now move pivot to its correct position, Swap $\text{arr}[i + 1] = \text{arr}[2]$ with $\text{arr}[4] = 40$.



Quick sort



Code Implementation

```
1  #include <stdio.h>
2
3  void swap(int* a, int* b);
4
5  // Partition function
6  int partition(int arr[], int low, int high) {
7
8      // Choose the pivot
9      int pivot = arr[high];
10
11     // Index of smaller element and indicates
12     // the right position of pivot found so far
13     int i = low - 1;
14
15     // Traverse arr[low..high] and move all smaller
16     // elements to the left side. Elements from low to
17     // i are smaller after every iteration
18     for (int j = low; j <= high - 1; j++) {
19         if (arr[j] < pivot) {
20             i++;
21             swap(&arr[i], &arr[j]);
22         }
23     }
24 }
```

```
24
25     // Move pivot after smaller elements and
26     // return its position
27     swap(&arr[i + 1], &arr[high]);
28     return i + 1;
29 }
30
31 // The QuickSort function implementation
32 void quickSort(int arr[], int low, int high) {
33     if (low < high) {
34
35         // pi is the partition return index of pivot
36         int pi = partition(arr, low, high);
37
38         // Recursion calls for smaller elements
39         // and greater or equals elements
40         quickSort(arr, low, pi - 1);
41         quickSort(arr, pi + 1, high);
42     }
43 }
44 }
```

```
45 void swap(int* a, int* b) {
46     int t = *a;
47     *a = *b;
48     *b = t;
49 }
50
51 int main() {
52     int arr[] = {10, 7, 8, 9, 1, 5};
53     int n = sizeof(arr) / sizeof(arr[0]);
54
55     quickSort(arr, 0, n - 1);
56     for (int i = 0; i < n; i++) {
57         printf("%d ", arr[i]);
58     }
59
60     return 0;
61 }
```

Complexity Analysis of Quick Sort

- **Time Complexity:**
 - **Best Case:** ($\Omega(n \log n)$), Occurs when the pivot element divides the array into two equal halves.
 - **Average Case** ($\theta(n \log n)$), On average, the pivot divides the array into two parts, but not necessarily equal.
 - **Worst Case:** ($O(n^2)$), Occurs when the smallest or largest element is always chosen as the pivot (e.g., sorted arrays).
- **Auxiliary Space:** $O(n)$, due to recursive call stack

Advantages of Quick Sort

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.
- It is Cache Friendly as we work on the same array to sort and do not copy data to any auxiliary array.
- Fastest general purpose algorithm for large data when stability is not required.
- It is tail recursive and hence all the tail call optimization can be done.

Disadvantages of Quick Sort

- It has a worst-case time complexity of $O(n^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

Application and uses of Quicksort

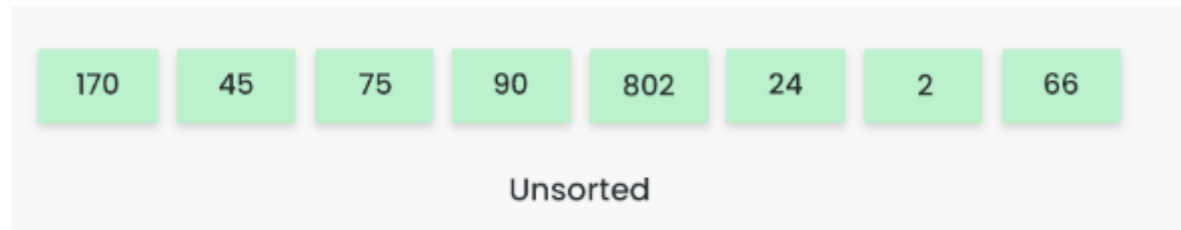
- ❑ **Commercial Computing** is used in various government and private organizations for the purpose of sorting various data like sorting files by name/date/price, sorting of students by their roll no., sorting of account profile by given id, etc as it is the fastest general purpose algorithm for large data.
- ❑ Since the two subarrays can be independently processed, we can easily do parallel processing.
- ❑ It is used almost everywhere where a **stable sort** is not needed. For stability, Merge Sort is typically used.
- ❑ Quicksort is a cache-friendly algorithm as it has a good locality of reference when used for arrays.
- ❑ It is **tail recursive** and hence all the tail call optimization can be done.
- ❑ It is an in-place sort that does not require any extra storage memory for storing elements (unlike Merge Sort that required extra space for merging)
- ❑ Numerical computations and in scientific research, for accuracy in calculations most of the efficiently developed algorithm uses priority queue and quick sort is used for sorting.

Radix Sort

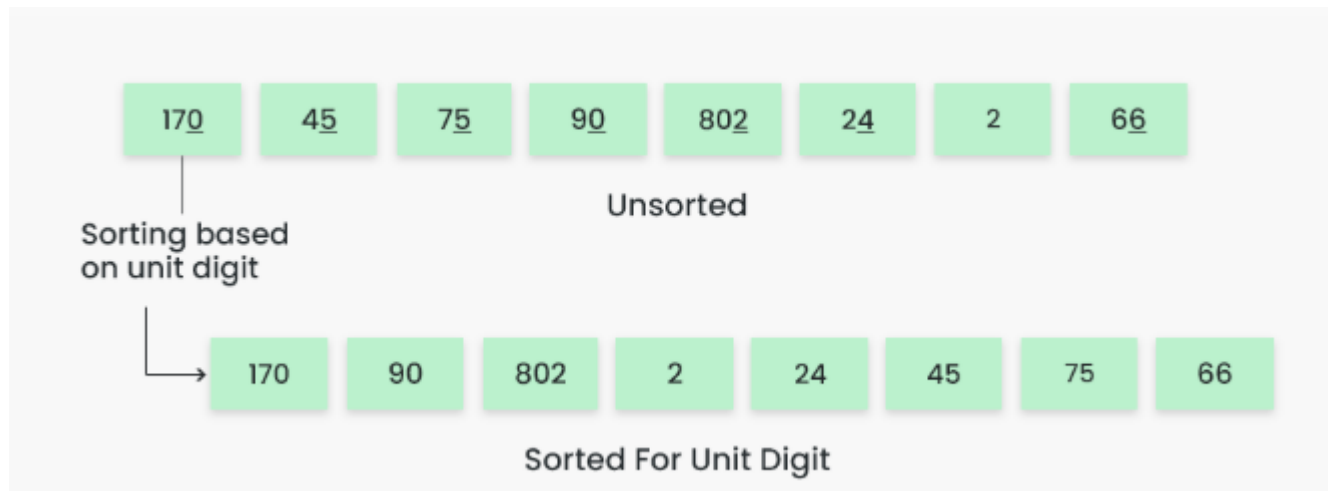
- **Radix Sort** is a linear sorting algorithm that sorts elements by processing them digit by digit. It is an efficient sorting algorithm for integers or strings with fixed-size keys.
- Rather than comparing elements directly, Radix Sort distributes the elements into buckets based on each digit's value. By repeatedly sorting the elements by their significant digits, from the least significant to the most significant, Radix Sort achieves the final sorted order.

How Radix sort works?

- *To perform radix sort on the array [170, 45, 75, 90, 802, 24, 2, 66], we follow these steps:*



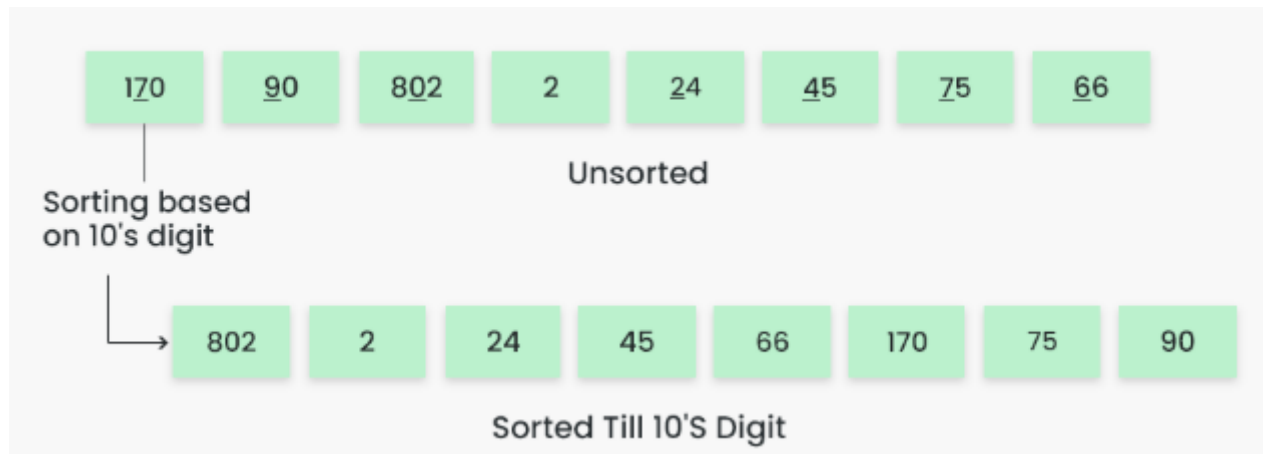
- **Step 1:** Find the largest element in the array, which is 802. It has three digits, so we will iterate three times, once for each significant place.
- **Step 2:** Sort the elements based on the unit place digits ($X=0$). We use a stable sorting technique, such as counting sort, to sort the digits at each significant place. It's important to understand that the default implementation of counting sort is unstable i.e. same keys can be in a different order than the input array. To solve this problem, We can iterate the input array in reverse order to build the output array. This strategy helps us to keep the same keys in the same order as they appear in the input array.
- **Sorting based on the unit place:**
 - Perform counting sort on the array based on the unit place digits.
 - The sorted array based on the unit place is [170, 90, 802, 2, 24, 45, 75, 66].



Step 3: Sort the elements based on the tens place digits.

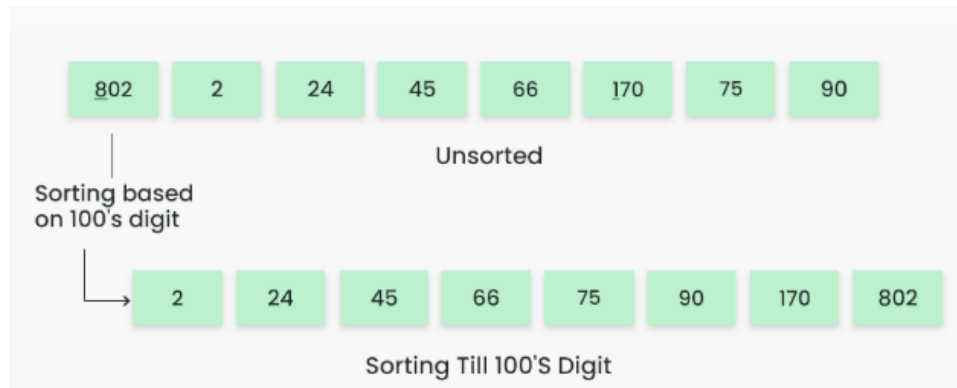
- **Sorting based on the tens place:**

- Perform counting sort on the array based on the tens place digits.
- The sorted array based on the tens place is [802, 2, 24, 45, 66, 170, 75, 90].



Step 4: Sort the elements based on the hundreds place digits.

- *Sorting based on the hundreds place:*
 - *Perform counting sort on the array based on the hundreds place digits.*
 - *The sorted array based on the hundreds place is [2, 24, 45, 66, 75, 90, 170, 802].*



Step 5: *The array is now sorted in ascending order.
The final sorted array using radix sort is [2, 24, 45, 66, 75, 90, 170, 802].*

Code

```
1  #include <stdio.h>
2
3  // A utility function to get the maximum
4  // value in arr[]
5  int getMax(int arr[], int n) {
6      int mx = arr[0];
7      for (int i = 1; i < n; i++)
8          if (arr[i] > mx)
9              mx = arr[i];
10     return mx;
11 }
12
13 // A function to do counting sort of arr[]
14 // according to the digit represented by exp
15 void countSort(int arr[], int n, int exp) {
16     int output[n]; // Output array
17     int count[10] = {0}; // Initialize count array as 0
18
19     // Store count of occurrences in count[]
20     for (int i = 0; i < n; i++)
21         count[(arr[i] / exp) % 10]++;
22
23     // Change count[i] so that count[i] now
24     // contains actual position of this digit
25     // in output[]
26     for (int i = 1; i < 10; i++)
27         count[i] += count[i - 1];
28
29     // Build the output array
30     for (int i = n - 1; i >= 0; i--) {
31         output[count[(arr[i] / exp) % 10] - 1] = arr[i];
32         count[(arr[i] / exp) % 10]--;
33     }
34 }
```

```
35     // Copy the output array to arr[],
36     // so that arr[] now contains sorted
37     // numbers according to current digit
38     for (int i = 0; i < n; i++)
39         arr[i] = output[i];
40 }
41
42 // The main function to sort arr[] of size
43 // n using Radix Sort
44 void radixSort(int arr[], int n) {
45
46     // Find the maximum number to know
47     // the number of digits
48     int m = getMax(arr, n);
49
50     // Do counting sort for every digit
51     // exp is 10^i where i is the current
52     // digit number
53     for (int exp = 1; m / exp > 0; exp *= 10)
54         countSort(arr, n, exp);
55 }
```

```
56
57 // A utility function to print an array
58 void printArray(int arr[], int n) {
59     for (int i = 0; i < n; i++)
60         printf("%d ", arr[i]);
61     printf("\n");
62 }
63
64 // Driver code
65 int main() {
66     int arr[] = {170, 45, 75, 90, 802, 24, 2, 66};
67     int n = sizeof(arr) / sizeof(arr[0]);
68
69     // Function call
70     radixSort(arr, n);
71     printArray(arr, n);
72     return 0;
73 }
```

Complexity Analysis of Radix Sort:

- **Time Complexity:**

- Radix sort is a non-comparative integer sorting algorithm that sorts data with integer keys by grouping the keys by the individual digits which share the same significant position and value. It has a time complexity of $O(d * (n + b))$, where d is the number of digits, n is the number of elements, and b is the base of the number system being used.
- In practical implementations, radix sort is often faster than other comparison-based sorting algorithms, such as quicksort or merge sort, for large datasets, especially when the keys have many digits. However, its time complexity grows linearly with the number of digits, and so it is not as efficient for small datasets.

- **Auxiliary Space:**

- Radix sort also has a space complexity of $O(n + b)$, where n is the number of elements and b is the base of the number system. This space complexity comes from the need to create buckets for each digit value and to copy the elements back to the original array after each digit has been sorted.

Stable and Unstable Sorting

- What is a stable sorting algorithm?

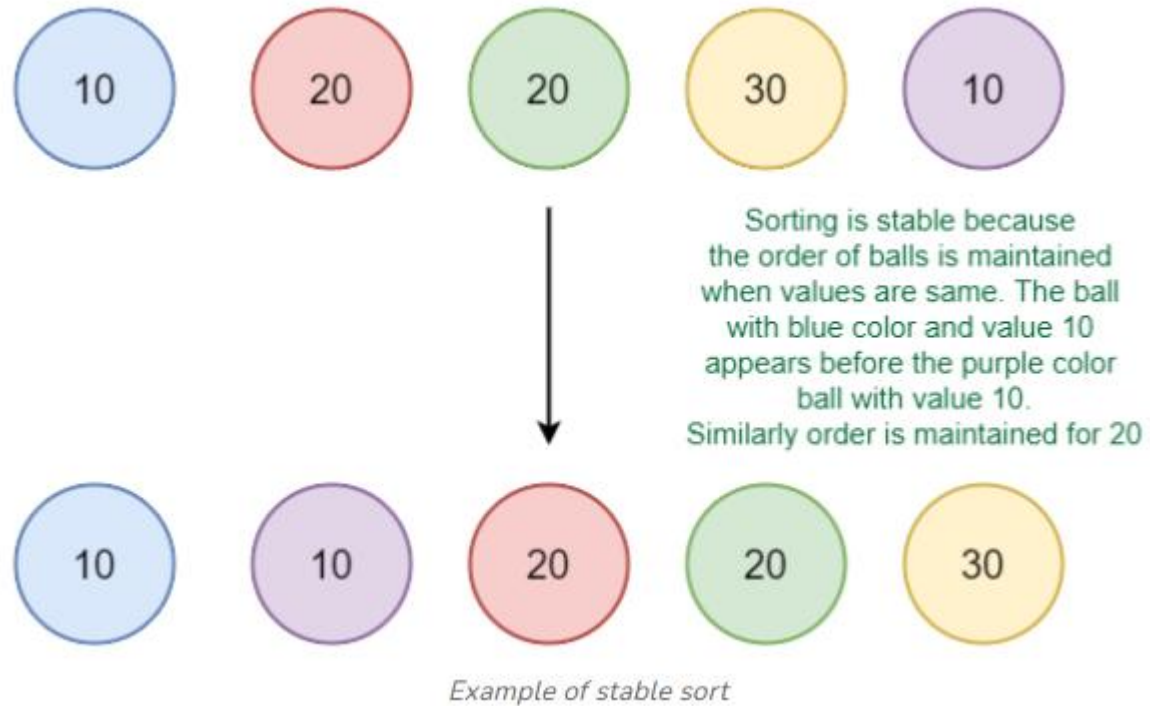
- *A sorting algorithm is said to be stable if two objects with equal keys appear in the same order in sorted output as they appear in the input data set*

Formally stability may be defined as, how the algorithm treats equal elements.

Let $A[]$ be an array, and let ' $<$ ' be a strict weak ordering on the elements of $A[]$. A sorting algorithm is stable if: $i < j$ and $A[i] \equiv A[j]$ implies $\pi(i) < \pi(j)$ where π is the sorting permutation (sorting moves $A[i]$ to position $\pi(i)$).

Informally, stability means that equivalent elements retain their relative positions, after sorting.

Stable Sort



Efficiency of **Sorting**

- Sorting algorithm efficiency is typically measured by its time complexity, which describes how the execution time grows as the input size increases.
- A good sorting algorithm generally aims for $O(n \log n)$ time complexity.
- Common sorting algorithms like Quick Sort, Merge Sort, and Heap Sort are known for their efficiency, often achieving this $O(n \log n)$ complexity.

Heap Sort

- **Heap sort** is a comparison-based sorting technique based on Binary Heap Data Structure. It can be seen as an optimization over selection sort where we first find the max (or min) element and swap it with the last (or first).
- We repeat the same process for the remaining elements.
- In Heap Sort, we use Binary Heap so that we can quickly find and move the max element in $O(\log n)$ instead of $O(n)$ and hence achieve the $O(n \log n)$ time complexity.

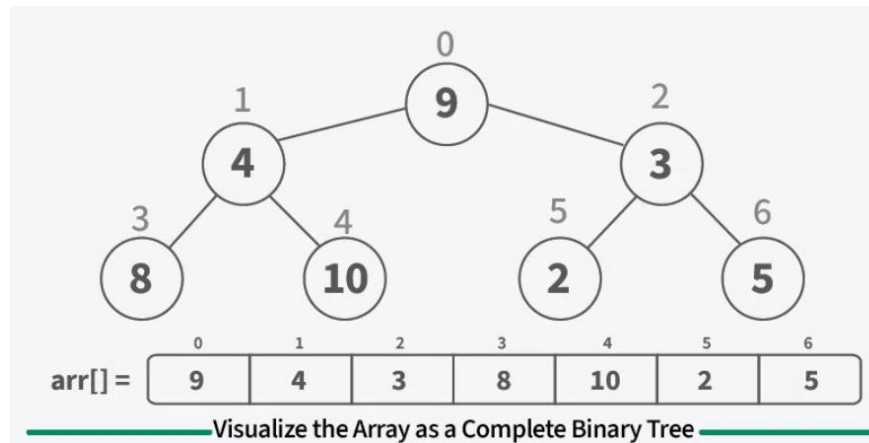
Heap Sort Algorithm

- First convert the array into a max heap using **heapify**, Please note that this happens in-place. The array elements are re-arranged to follow heap properties. Then one by one delete the root node of the Max-heap and replace it with the last node and **heapify**. Repeat this process while size of heap is greater than 1.
- Rearrange array elements so that they form a Max Heap.
- Repeat the following steps until the heap contains only one element:
 - Swap the root element of the heap (which is the largest element in current heap) with the last element of the heap.
 - Remove the last element of the heap (which is now in the correct position). We mainly reduce heap size and do not remove element from the actual array.
 - Heapify the remaining elements of the heap.
- Finally we get sorted array.

Detailed Working of Heap Sort

Step 1: Treat the Array as a Complete Binary Tree

- We first need to visualize the array as a **complete binary tree**. For an array of size n , the root is at index 0 , the left child of an element at index i is at $2i + 1$, and the right child is at $2i + 2$.

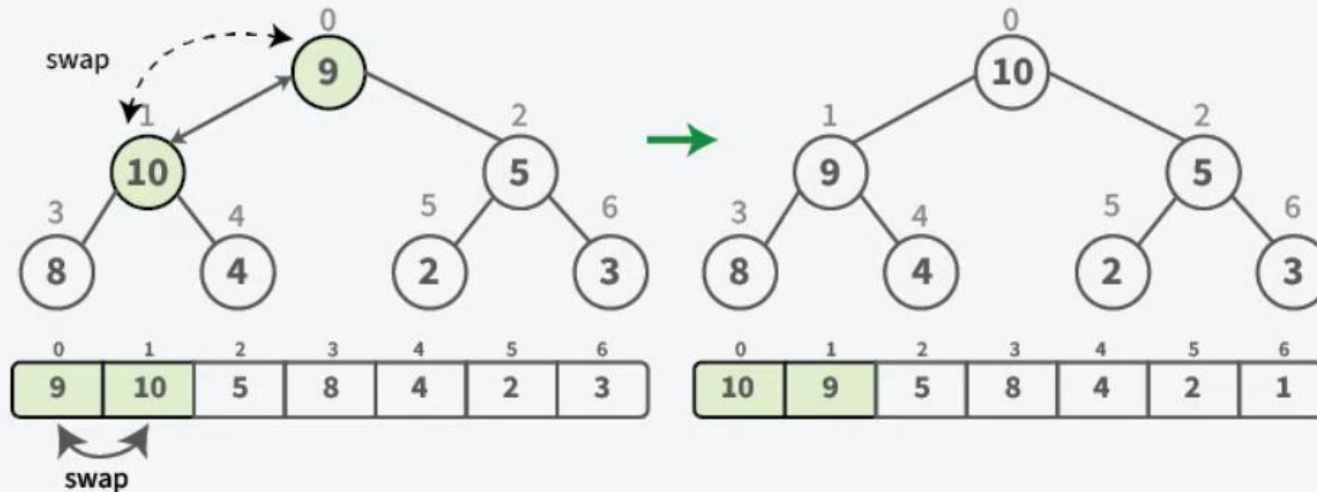


Detailed Working of Heap Sort

Step 2: Build a Max Heap

07
Step

Node 9 is smaller than its child node (10), so swap it with the larger child to maintain the property that the parent should be larger than its children.

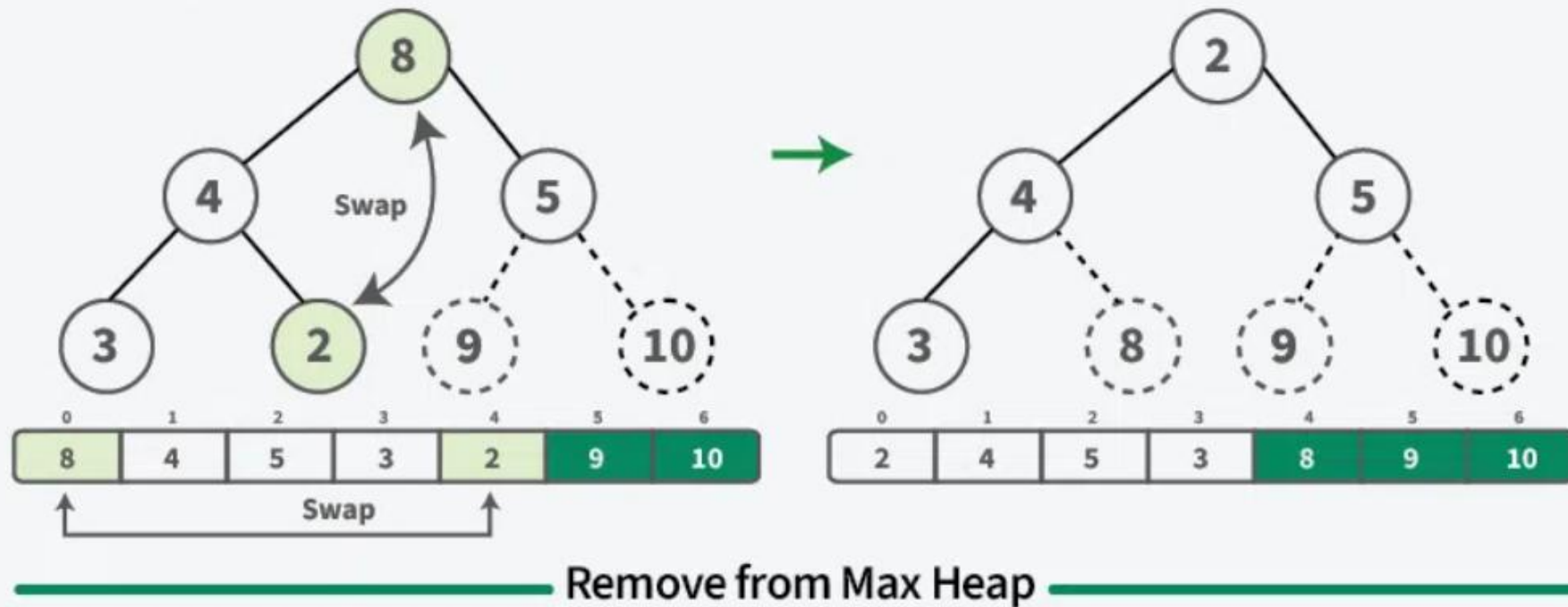


Heapify Binary Tree

Step 3: Sort the array by placing largest element at end of unsorted array.

06
Step

Now, we have a max heap. Swap the maximum element (8) with the last element (2) in the unsorted array, then decrease the heap size by one (ignoring the third last element as it's now sorted).



Complexity Analysis of Heap Sort

- **Time Complexity:** $O(n \log n)$
- **Auxiliary Space:** $O(\log n)$, due to the recursive call stack. However, auxiliary space can be $O(1)$ for iterative implementation.
- **Important points about Heap Sort**
 - Heap sort is an in-place algorithm.
 - Its typical implementation is not stable but can be made stable (See [this](#))
 - Typically 2-3 times slower than well-implemented [QuickSort](#). The reason for slowness is a lack of locality of reference.

Advantages of **Heap Sort**

- ❑ **Efficient Time Complexity:** Heap Sort has a time complexity of $O(n \log n)$ in all cases. This makes it efficient for sorting large datasets. The **log n** factor comes from the height of the binary heap, and it ensures that the algorithm maintains good performance even with a large number of elements.
- ❑ **Memory Usage:** Memory usage can be minimal (by writing an iterative `heapify()` instead of a recursive one). So apart from what is necessary to hold the initial list of items to be sorted, it needs no additional memory space to work
- ❑ **Simplicity:** It is simpler to understand than other equally efficient sorting algorithms because it does not use advanced computer science concepts such as recursion.

Disadvantages of Heap Sort

- ❑ **Costly:** Heap sort is costly as the constants are higher compared to merge sort even if the time complexity is $O(n \log n)$ for both.
- ❑ **Unstable:** Heap sort is unstable. It might rearrange the relative order.
- ❑ **Inefficient:** Heap Sort is not very efficient because of the high constants in the time complexity.

Bucket Sort

□ **Bucket sort** is a sorting technique that involves dividing elements into various groups, or buckets. These buckets are formed by uniformly distributing the elements. Once the elements are divided into buckets, they can be sorted using any other sorting algorithm. Finally, the sorted elements are gathered together in an ordered fashion.

- **Bucket Sort Algorithm:**

- Create n empty buckets (Or lists) and do the following for every array element $arr[i]$.
- Insert $arr[i]$ into $bucket[n * array[i]]$
- Sort individual buckets using insertion sort.
- Concatenate all sorted buckets.

How does **Bucket Sort** Works?

- ❑ To apply bucket sort on the input array [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68], we follow these steps:
- ❑ **Step 1:** Create an array of size 10, where each slot represents a bucket.



How does **Bucket Sort** Works?

- **Step 2:** Insert elements into the buckets from the input array based on their range.
 - ❑ Inserting elements into the buckets:
 - ❑ Take each element from the input array.
 - ❑ Multiply the element by the size of the bucket array (10 in this case). For example, for element 0.23, we get $0.23 * 10 = 2.3$.
 - ❑ Convert the result to an integer, which gives us the bucket index. In this case, 2.3 is converted to the integer 2.
 - ❑ Insert the element into the bucket corresponding to the calculated index.
 - ❑ Repeat these steps for all elements in the input array.

How does **Bucket Sort** Works?

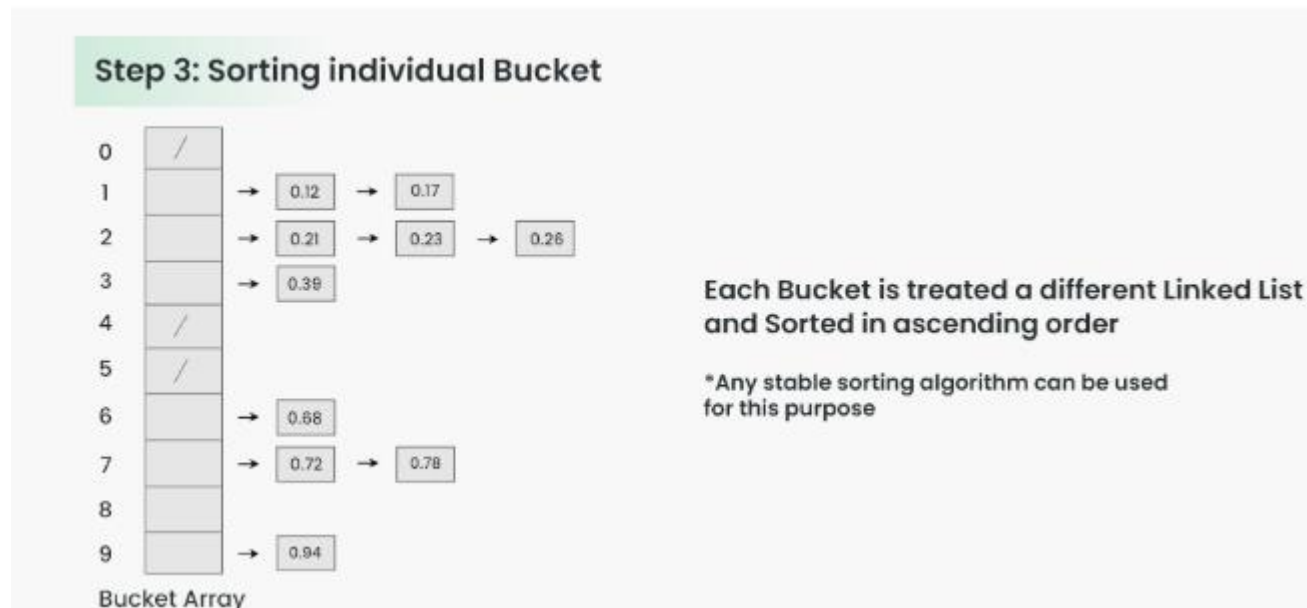
step 2 : Inserting Array elements into respective buckets



How does **Bucket Sort** Works?

- **Step 3:** Sort the elements within each bucket. In this example, we use quicksort (or any stable sorting algorithm) to sort the elements within each bucket.
- Sorting the elements within each bucket:
 - Apply a stable sorting algorithm (e.g., Bubble Sort, Merge Sort) to sort the elements within each bucket.
 - The elements within each bucket are now sorted.

How does **Bucket Sort** Works?

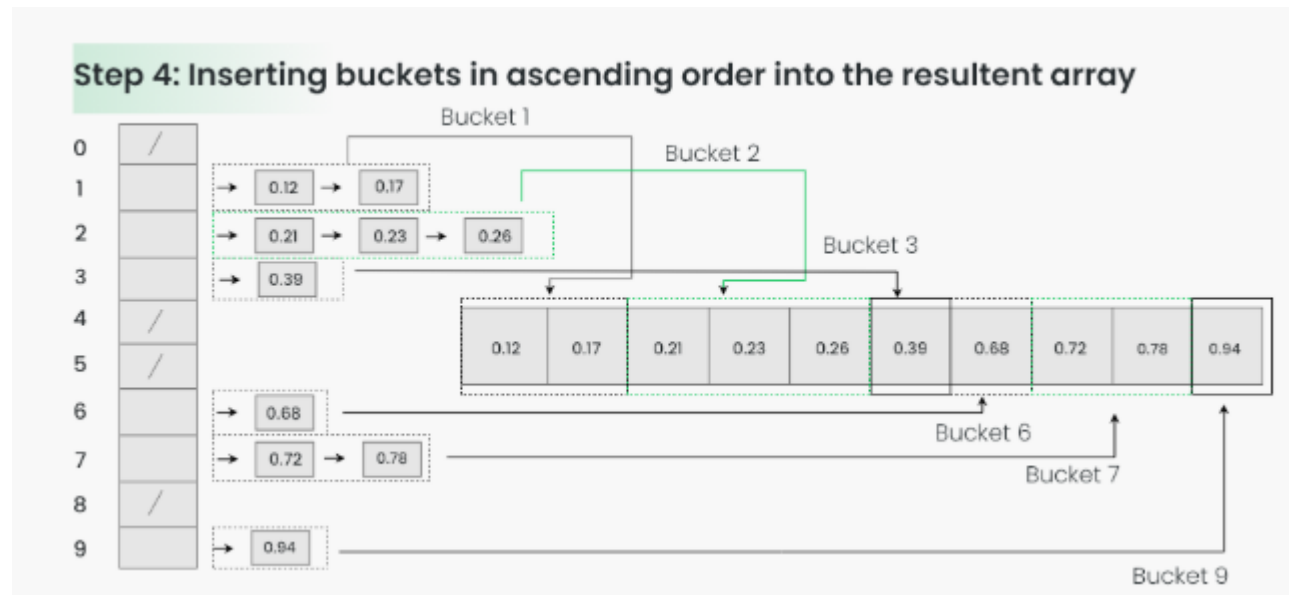


How does **Bucket Sort** Works?

Step 4: Gather the elements from each bucket and put them back into the original array.

- Gathering elements from each bucket:
- Iterate through each bucket in order.
- Insert each individual element from the bucket into the original array.
- Once an element is copied, it is removed from the bucket.
- Repeat this process for all buckets until all elements have been gathered.

How does **Bucket Sort** Works?



How does **Bucket Sort** Works?

- **Step 5:** The original array now contains the sorted elements.
- The final sorted array using bucket sort for the given input is [0.12, 0.17, 0.21, 0.23, 0.26, 0.39, 0.68, 0.72, 0.78, 0.94].

Step 5 : Return the Sorted Array

0.12	0.17	0.21	0.23	0.26	0.39	0.68	0.72	0.78	0.94
------	------	------	------	------	------	------	------	------	------

Complexity Analysis of Bucket Sort Algorithm:

- **Worst Case Time Complexity:** $O(n^2)$ The worst case happens when one bucket gets all the elements. In this case, we will be running insertion sort on all items which will make the time complexity as $O(n^2)$. We can reduce the worst case time complexity to $O(n \log n)$ by using a $O(n \log n)$ algorithm like Merge Sort or Heap Sort to sort the individual buckets, but that will improve the algorithm time for cases when buckets have small number of items as insertion sort works better for small arrays.
- **Best Case Time Complexity :** $O(n + k)$ The best case happens when every bucket gets equal number of elements. In this case every call to insertion sort will take constant time as the number of items in every bucket would be constant (Assuming that k is linearly proportional to n).
- **Auxiliary Space:** $O(n+k)$

Analysis of different **sorting** techniques

Sorting Algorithm	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	$\Omega(N)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$\Omega(N^2)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Insertion Sort	$\Omega(N)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Merge Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N \log N)$	$O(N)$
Heap Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N \log N)$	$O(1)$
Quick Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N^2)$	$O(\log N)$
Radix Sort	$\Omega(N k)$	$\Theta(N k)$	$O(N k)$	$O(N + k)$
Count Sort	$\Omega(N + k)$	$\Theta(N + k)$	$O(N + k)$	$O(k)$
Bucket Sort	$\Omega(N + k)$	$\Theta(N + k)$	$O(N^2)$	$O(N)$

Analysis of different **sorting** techniques

Sorting Algorithm	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	$\Omega(N)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$\Omega(N^2)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Insertion Sort	$\Omega(N)$	$\Theta(N^2)$	$O(N^2)$	$O(1)$
Merge Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N \log N)$	$O(N)$
Heap Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N \log N)$	$O(1)$
Quick Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N^2)$	$O(\log N)$
Radix Sort	$\Omega(N k)$	$\Theta(N k)$	$O(N k)$	$O(N + k)$
Count Sort	$\Omega(N + k)$	$\Theta(N + k)$	$O(N + k)$	$O(k)$
Bucket Sort	$\Omega(N + k)$	$\Theta(N + k)$	$O(N^2)$	$O(N)$

Searching Algorithm

- **Searching algorithms** are essential tools in computer science used to locate specific items within a collection of data. In this section, we are mainly going to focus upon searching in an array. When we search an item in an array, there are two most common algorithms used based on the type of input array.
- **Linear Search** : It is used for an unsorted array. It mainly does one by one comparison of the item to be search with array elements. It takes linear or $O(n)$ Time.
- **Binary Search** : It is used for a sorted array. It mainly compares the array's middle element first and if the middle element is same as input, then it returns. Otherwise it searches in either left half or right half based on comparison result (Whether the mid element is smaller or greater). This algorithm is faster than linear search and takes $O(\log n)$ time.

Linear Search Algorithm

- **Sequential search**, also known as linear search, is a simple algorithm for finding a specific item within a list.
- It works by examining each element of the list one by one in order until a match is found or the entire list has been checked.
- Given an array, **arr** of n integers, and an integer element **x**, find whether element **x** is present in the array. Return the index of the first occurrence of **x** in the array, or -1 if it doesn't exist.
- **Input:** `arr[] = [1, 2, 3, 4]`, `x = 3`
Output: 2
Explanation: There is one test case with array as [1, 2, 3 4] and element to be searched as 3. Since 3 is present at index 2, the output is 2.

For example

03 Step | Compare the key with the 3rd element. Key is found so stop the search.

Key
30 Equal

0	1	2	3	4	5	6	7	8
10	50	30	70	80	60	20	90	40

Cur

Linear Search Algorithm

< ▶ >

3 / 3

Code

```
1 // C code to linearly search x in arr[.
2
3 #include <stdio.h>
4
5 int search(int arr[], int N, int x)
6 {
7     for (int i = 0; i < N; i++)
8         if (arr[i] == x)
9             return i;
10    return -1;
11 }
12
13 // Driver code
14 int main(void)
15 {
16     int arr[] = { 2, 3, 4, 10, 40 };
17     int x = 10;
18     int N = sizeof(arr) / sizeof(arr[0]);
19
20     // Function call
21     int result = search(arr, N, x);
22     (result == -1)
23         ? printf("Element is not present in array")
24         : printf("Element is present at index %d", result);
25     return 0;
26 }
```

Time and Space Complexity of Linear Search Algorithm:

- **Time Complexity:**
 - **Best Case:** In the best case, the key might be present at the first index. So the best case complexity is $O(1)$
 - **Worst Case:** In the worst case, the key might be present at the last index i.e., opposite to the end from which the search has started in the list. So the worst-case complexity is $O(N)$ where N is the size of the list.
 - **Average Case:** $O(N)$
- **Auxiliary Space:** $O(1)$ as except for the variable to iterate through the list, no other variable is used.

Applications of Linear Search Algorithm:

- **Unsorted Lists:** When we have an unsorted array or list, linear search is most commonly used to find any element in the collection.
- **Small Data Sets:** Linear Search is preferred over binary search when we have small data sets with
- **Searching Linked Lists:** In linked list implementations, linear search is commonly used to find elements within the list. Each node is checked sequentially until the desired element is found.
- **Simple Implementation:** Linear Search is much easier to understand and implement as compared to Binary Search or Ternary Search.

Advantages and Disadvantage

Advantage:

- Linear search can be used irrespective of whether the array is sorted or not. It can be used on arrays of any data type.
- Does not require any additional memory.
- It is a well-suited algorithm for small datasets.

Disadvantages :

- Linear search has a time complexity of $O(N)$, which in turn makes it slow for large datasets.
- Not suitable for large arrays.

Binary Search Algorithm

- **Binary Search Algorithm** is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half.
- The idea of binary search is to use the information that the array is sorted and reduce the time complexity to $O(\log N)$.
- **Conditions to apply Binary Search Algorithm in a Data Structure**
 - To apply Binary Search algorithm:
 - The data structure must be sorted.
 - Access to any element of the data structure should take constant time.

Binary Search Algorithm

- Divide the search space into two halves by **finding the middle index “mid”**.
- Compare the middle element of the search space with the **key**.
- If the **key** is found at middle element, the process is terminated.
- If the **key** is not found at middle element, choose which half will be used as the next search space.
 - If the **key** is smaller than the middle element, then the **left** side is used for next search.
 - If the **key** is larger than the middle element, then the **right** side is used for next search.
- This process is continued until the **key** is found or the total search space is exhausted.

How does Binary Search Algorithm work?

- To understand the working of binary search, consider the following illustration:
- Consider an array `arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`, and the **target = 23**.

Step 3 | If the key matches the value of the mid element, the element is found and stop search.

Key
23

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

Low = 5 High = 6
Mid = 5

< ▶ >

4 / 4

Code

```
1 // C program to implement iterative Binary Search
2 #include <stdio.h>
3
4 // An iterative binary search function.
5 int binarySearch(int arr[], int low, int high, int x)
6 {
7     while (low <= high) {
8         int mid = low + (high - low) / 2;
9
10        // Check if x is present at mid
11        if (arr[mid] == x)
12            return mid;
13
14        // If x greater, ignore left half
15        if (arr[mid] < x)
16            low = mid + 1;
17
18        // If x is smaller, ignore right half
19        else
20            high = mid - 1;
21    }
22
23    // If we reach here, then element was not present
24    return -1;
25 }
```

```
26
27 // Driver code
28 int main(void)
29 {
30     int arr[] = { 2, 3, 4, 10, 40 };
31     int n = sizeof(arr) / sizeof(arr[0]);
32     int x = 10;
33     int result = binarySearch(arr, 0, n - 1, x);
34     if(result == -1) printf("Element is not present in array");
35     else printf("Element is present at index %d",result);
36
37 }
```

Time Complexity: $O(\log N)$
Auxiliary Space: $O(1)$

Complexity Analysis of Binary Search Algorithm

- **Time Complexity:**
 - Best Case: $O(1)$
 - Average Case: $O(\log N)$
 - Worst Case: $O(\log N)$
- **Auxiliary Space:** $O(1)$, If the recursive call stack is considered then the auxiliary space will be $O(\log N)$.

Applications of Binary Search Algorithm

- Binary search can be used as a building block for more complex algorithms used in machine learning, such as algorithms for training neural networks or finding the optimal hyperparameters for a model.
- It can be used for searching in computer graphics such as algorithms for ray tracing or texture mapping.
- It can be used for searching a database.

Binary Vs Linear Search

Feature	Linear Search	Binary Search
Definition	Checks each element one by one	Divides the sorted list in half repeatedly
Data Requirement	Works on unsorted or sorted data	Works only on sorted data
Time Complexity		
- Best Case	$O(1)$	$O(1)$
- Average Case	$O(n)$	$O(\log n)$
- Worst Case	$O(n)$	$O(\log n)$
Space Complexity	$O(1)$	$O(1)$ (iterative) / $O(\log n)$ (recursive)
Ease of Implementation	Very easy	Slightly more complex
Performance	Slower on large datasets	Faster on large sorted datasets
Use Case	Small or unsorted datasets	Large and sorted datasets

Presented By: Saroj Bhandari



Thank you !!!